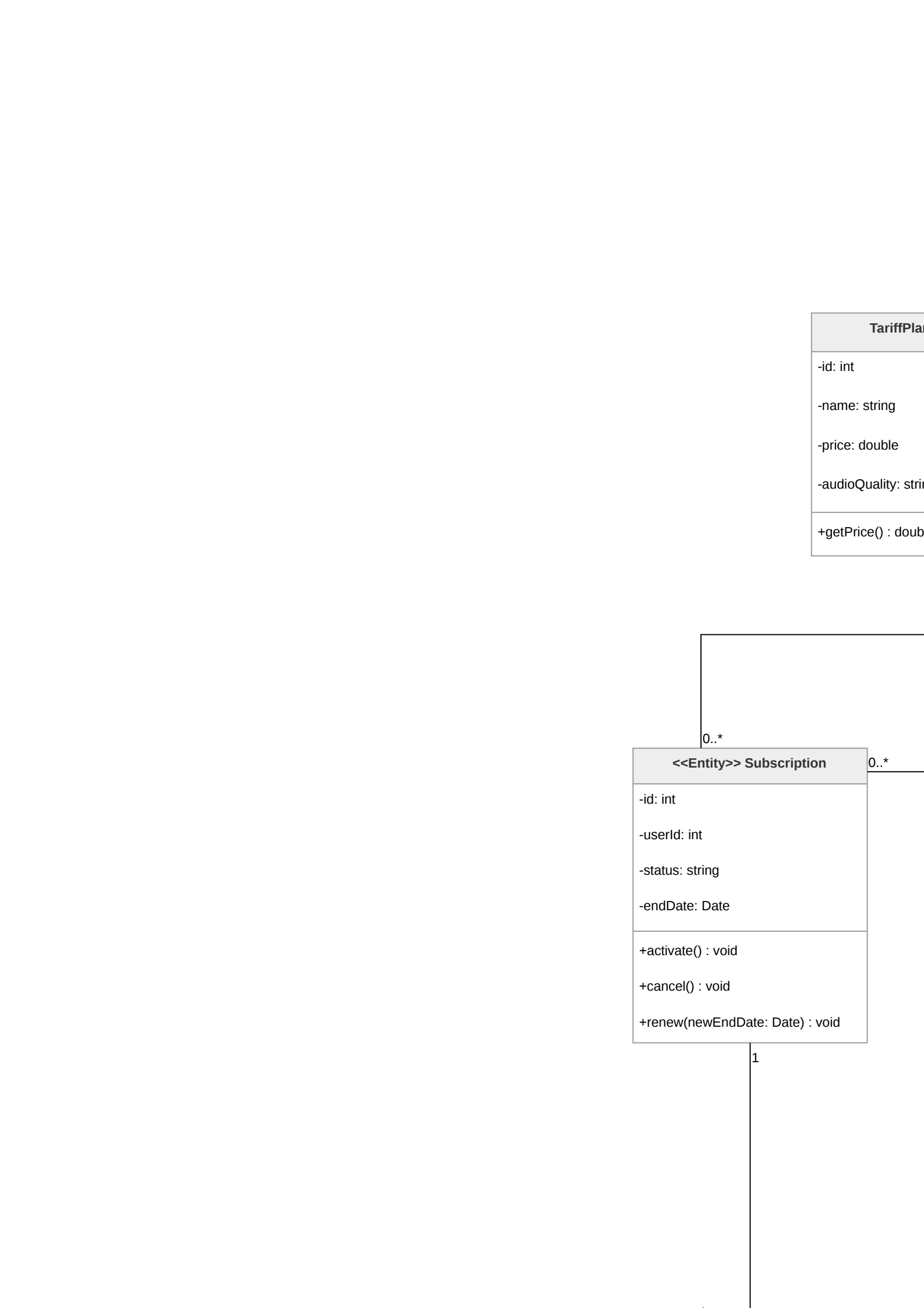
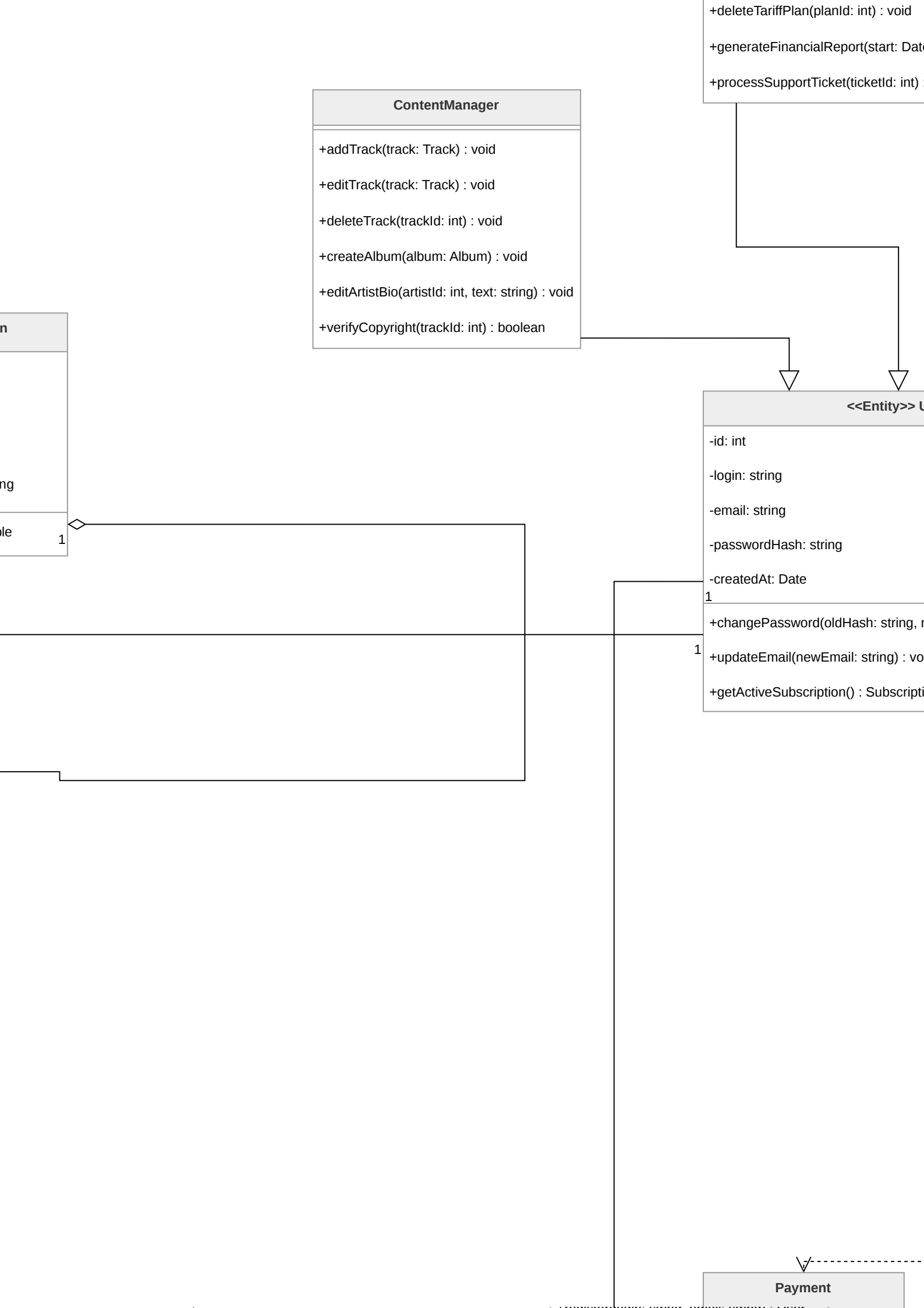
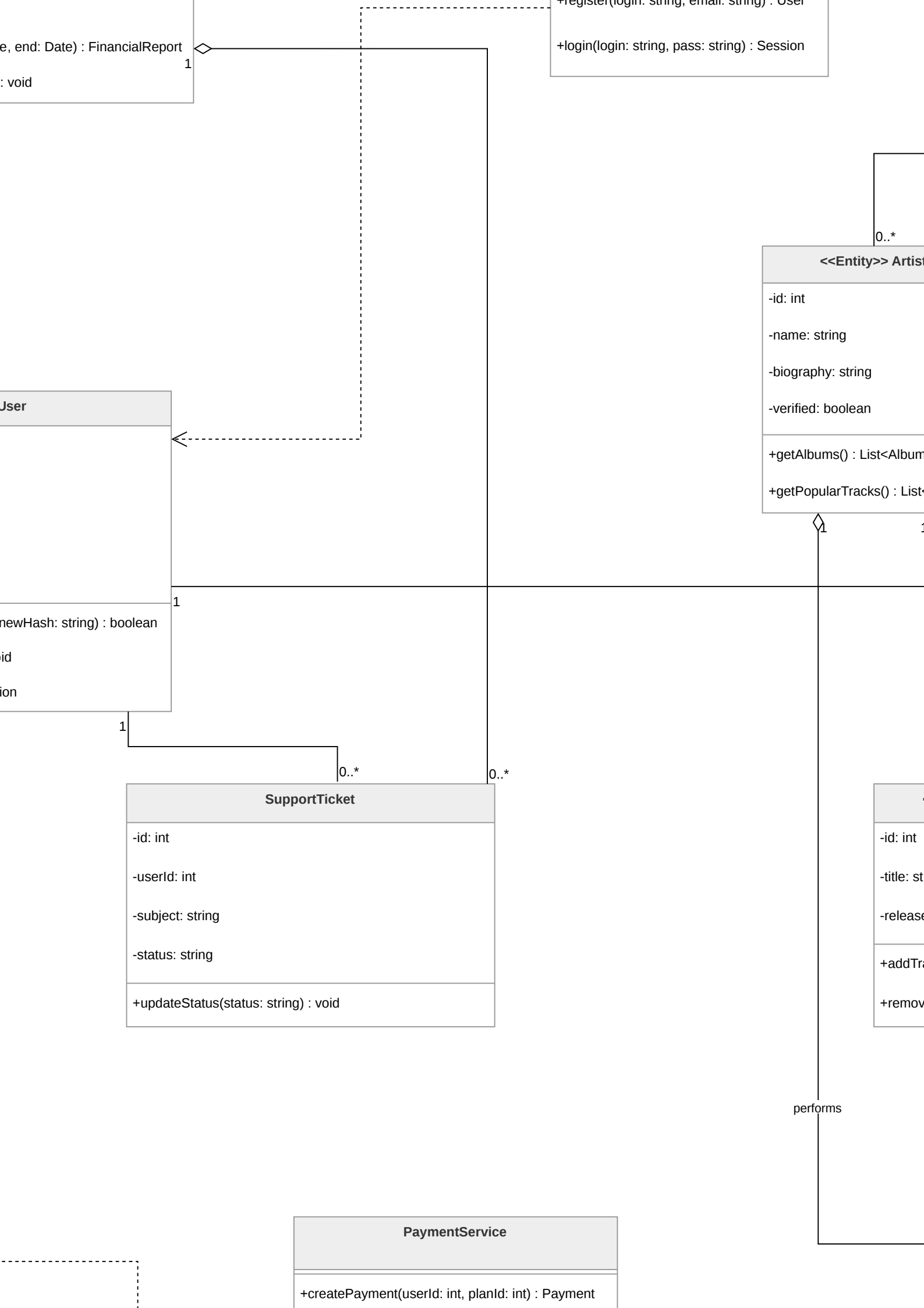
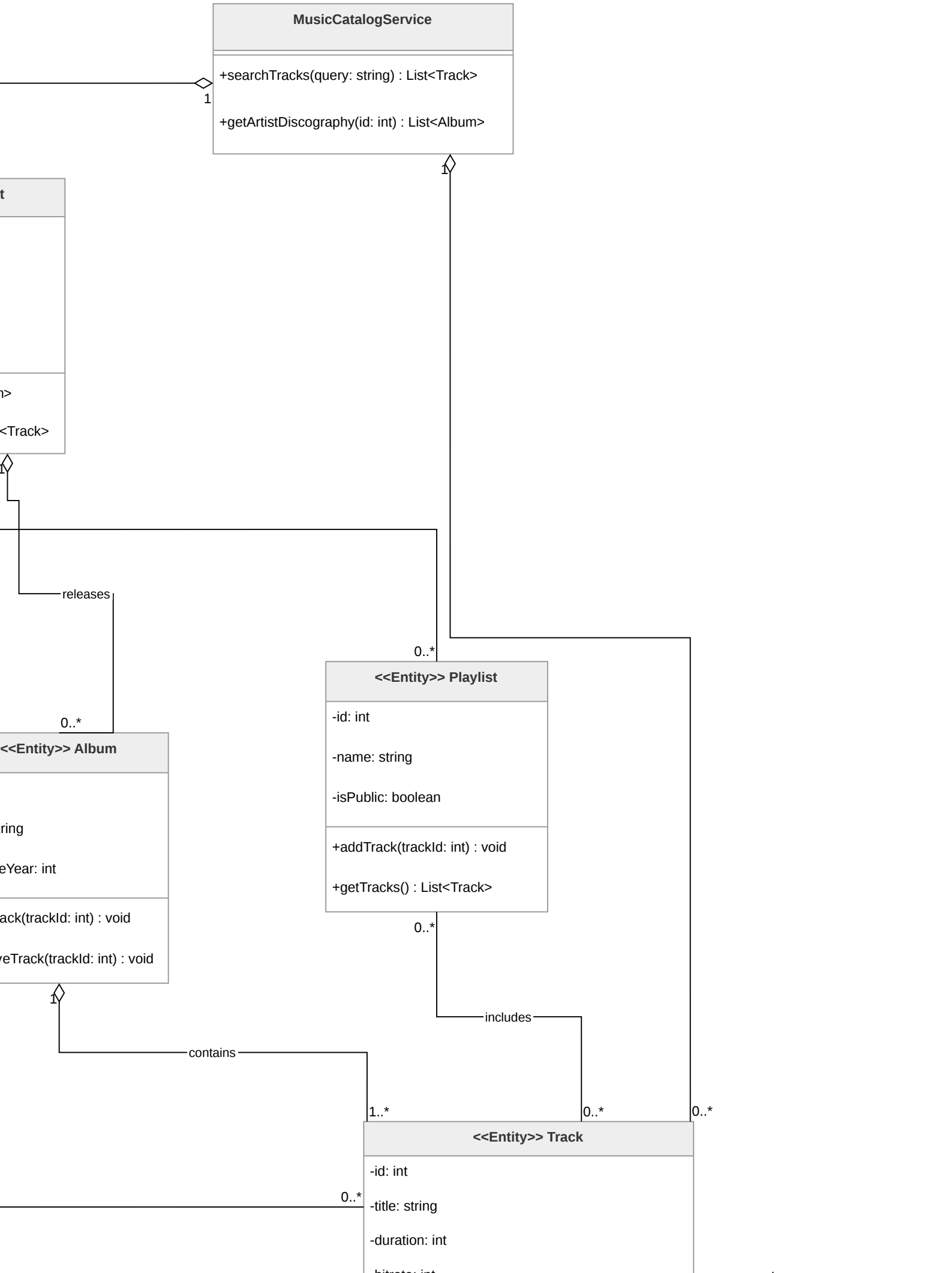

Admi
+blockUser(userId: int) : void
+unblockUser(userId: int) : void
+addTariffPlan(plan: TariffPlan) : voi


n
d
AuthService


TariffPla
-id: int
-name: string
-price: double
-audioQuality: stri
+getPrice() : doub
0..*
<<Entity>> Subscription
0..*
-id: int
-userId: int
-status: string
-endDate: Date
+activate() : void
+cancel() : void
+renew(newEndDate: Date) : void
1

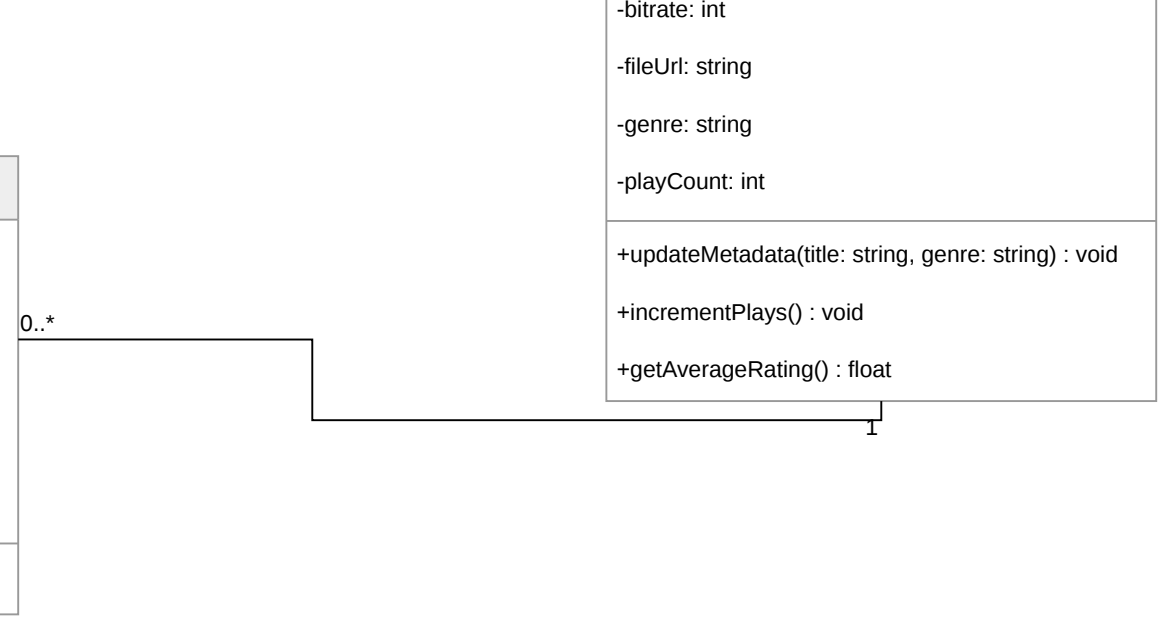

+deleteTariffPlan(planId: int) : void
+generateFinancialReport(start: Dat
+processSupportTicket(ticketId: int)
ContentManager
+addTrack(track: Track) : void
+editTrack(track: Track) : void
+deleteTrack(trackId: int) : void
+createAlbum(album: Album) : void
+editArtistBio(artistId: int, text: string) : void
+verifyCopyright(trackId: int) : boolean
n
ng
ble
1
<<Entity>> U
-id: int
-login: string
-email: string
-passwordHash: string
-createdAt: Date
1
+changePassword(oldHash: string,
1
+updateEmail(newEmail: string) : vo
+getActiveSubscription() : Subscripti
Payment


+register(login: string, email: string) : User
+login(login: string, pass: string) : Session
e, end: Date) : FinancialReport
: void
1
0..*
<<Entity>> Artist
-id: int
-name: string
-biography: string
-verified: boolean
+getAlbums() : List<Album
+getPopularTracks() : List
User
newHash: string) : boolean
id
ion
1
1
0..*
0..*
SupportTicket
-id: int
-userId: int
-subject: string
-status: string
+updateStatus(status: string) : void
-id: int
-title: st
-release
+addTra
+remov
performs
PaymentService
+createPayment(userId: int, planId: int) : Payment

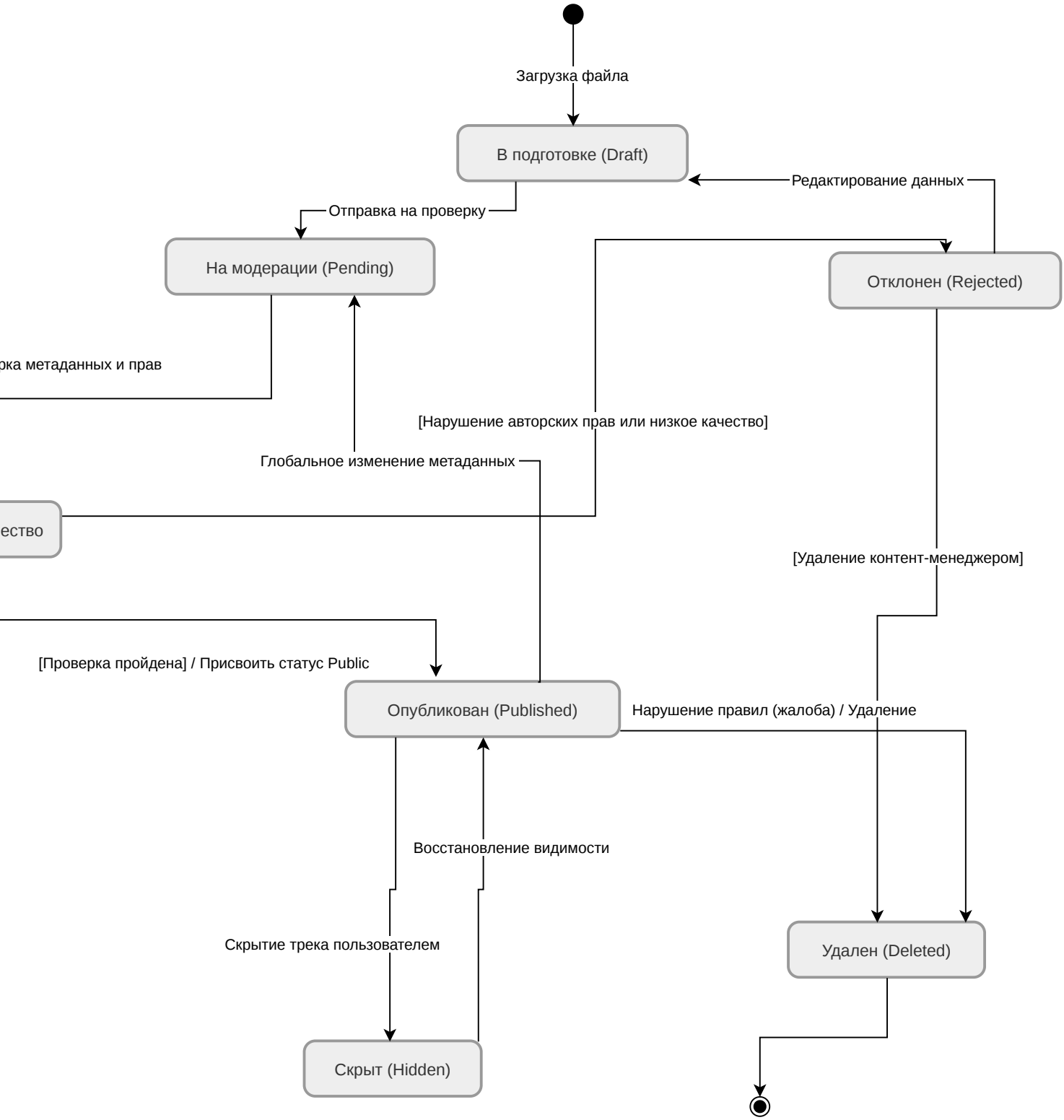

MusicCatalogService
+searchTracks(query: string) : List<Track>
+getArtistDiscography(id: int) : List<Album>
1
1
releases
0..*
0..*
<<Entity>> Playlist
-id: int
-name: string
-isPublic: boolean
+addTrack(trackId: int) : void
+getTracks() : List<Track>
0..*
includes
contains
1..*
0..*
0..*
<<Entity>> Track
-id: int
-title: string
-duration: int
0..*

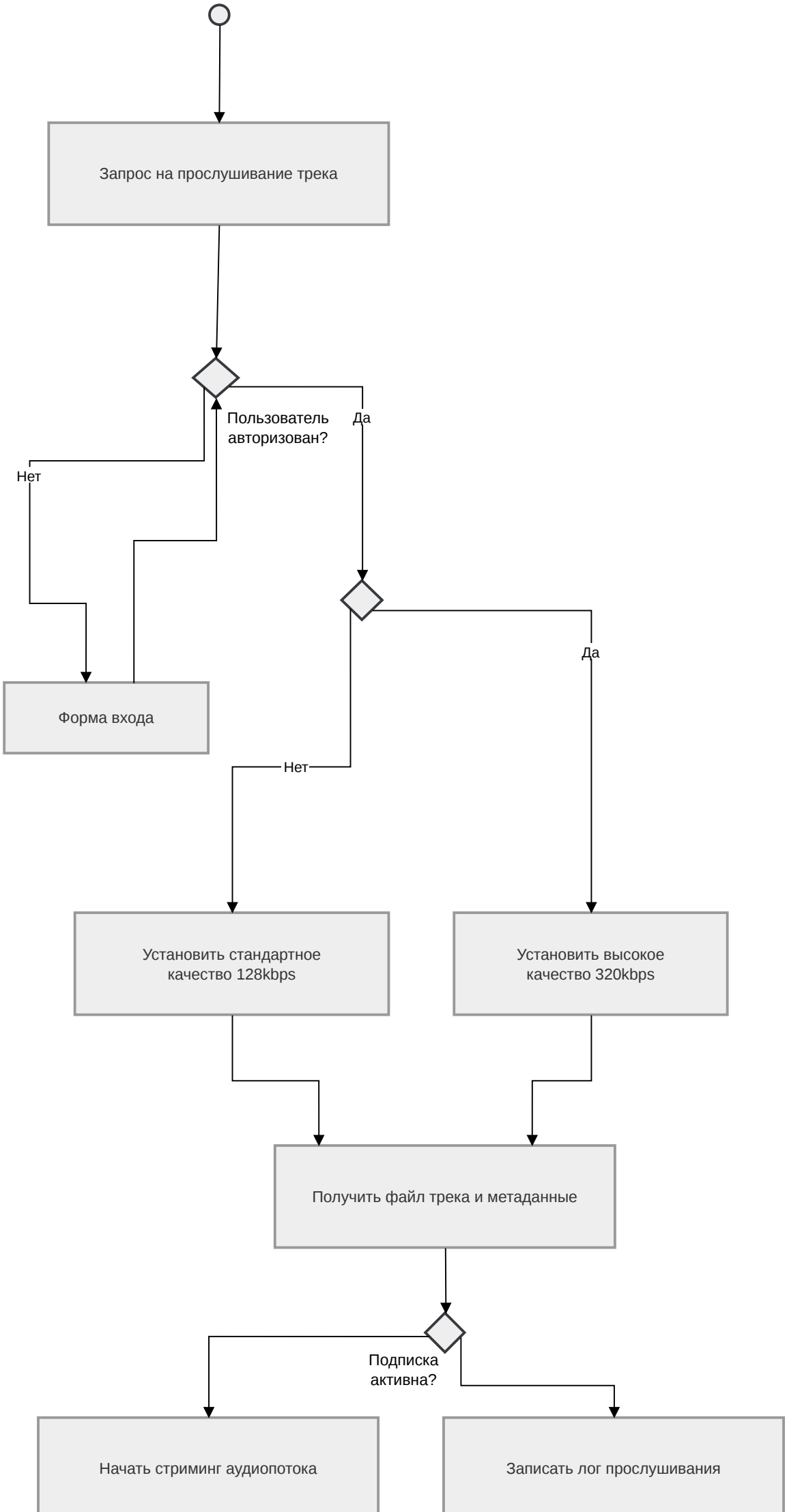

0..*
-id: int
-userId: int
-amount: double
-status: string
+confirm(txId: string) : void
+fail(reason: string) : void

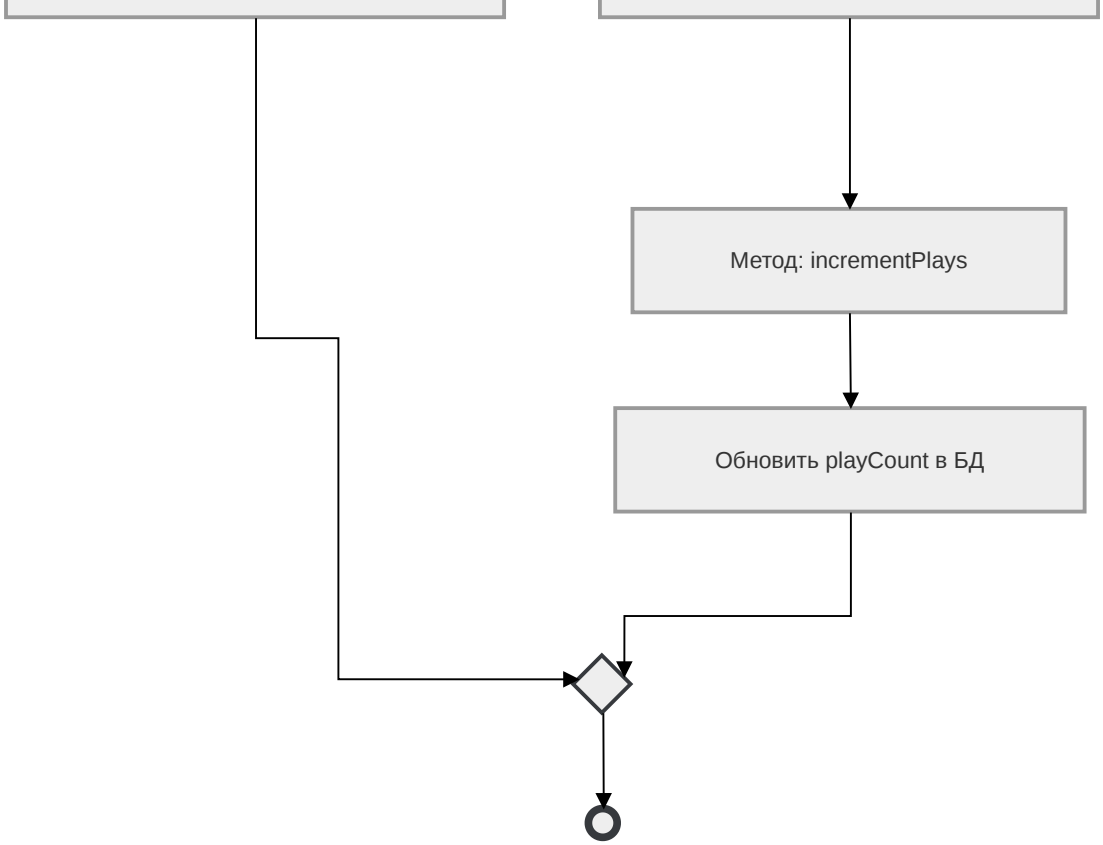

Review
-id: int
-userId: int
-trackId: int
-text: string
-rating: int
+editText(newText: string) : void
0..*

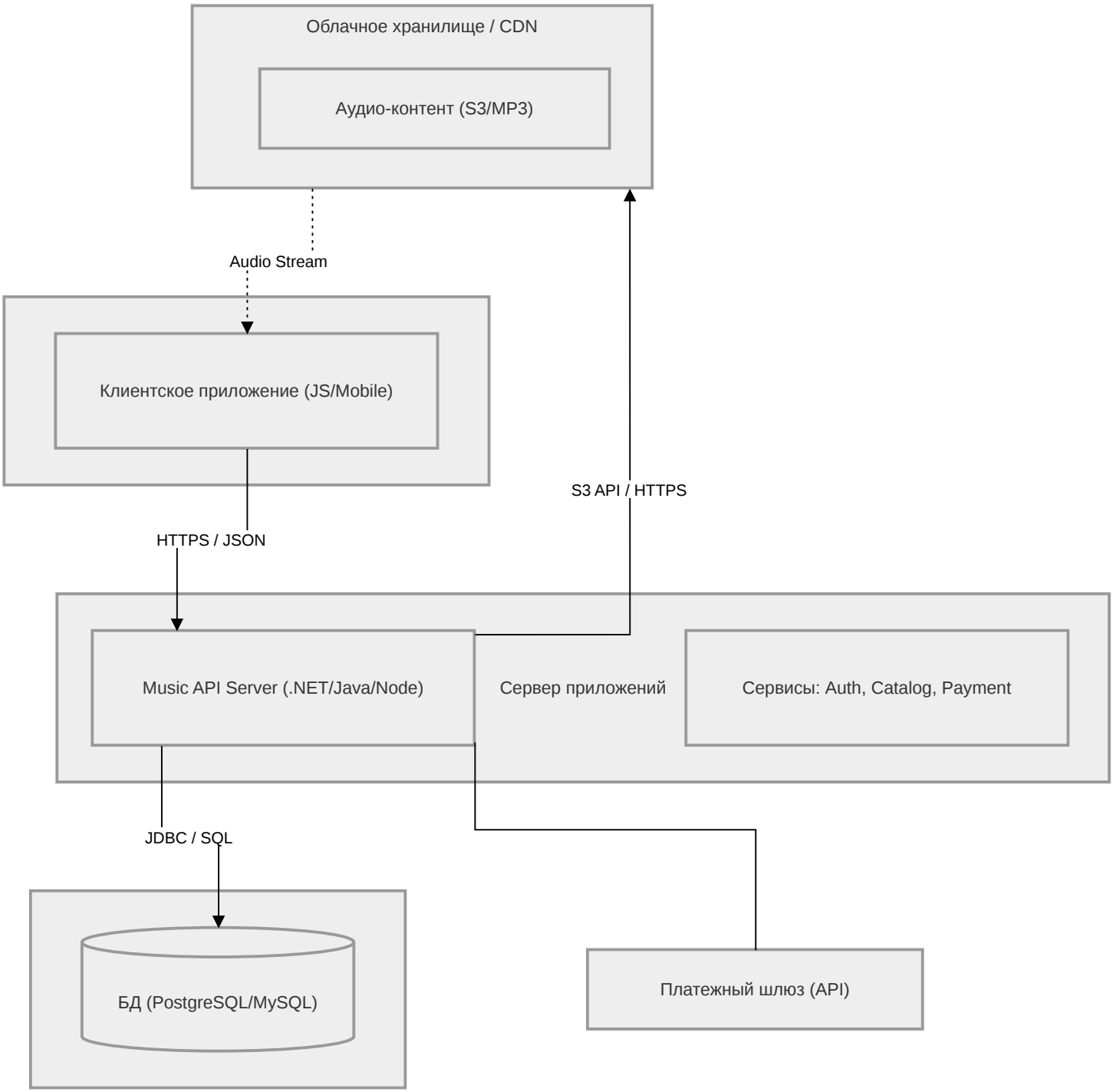

-bitrate: int
-fileUrl: string
-genre: string
-playCount: int
+updateMetadata(title: string, genre: string) : void
+incrementPlays() : void
+getAverageRating() : float
0..*
1

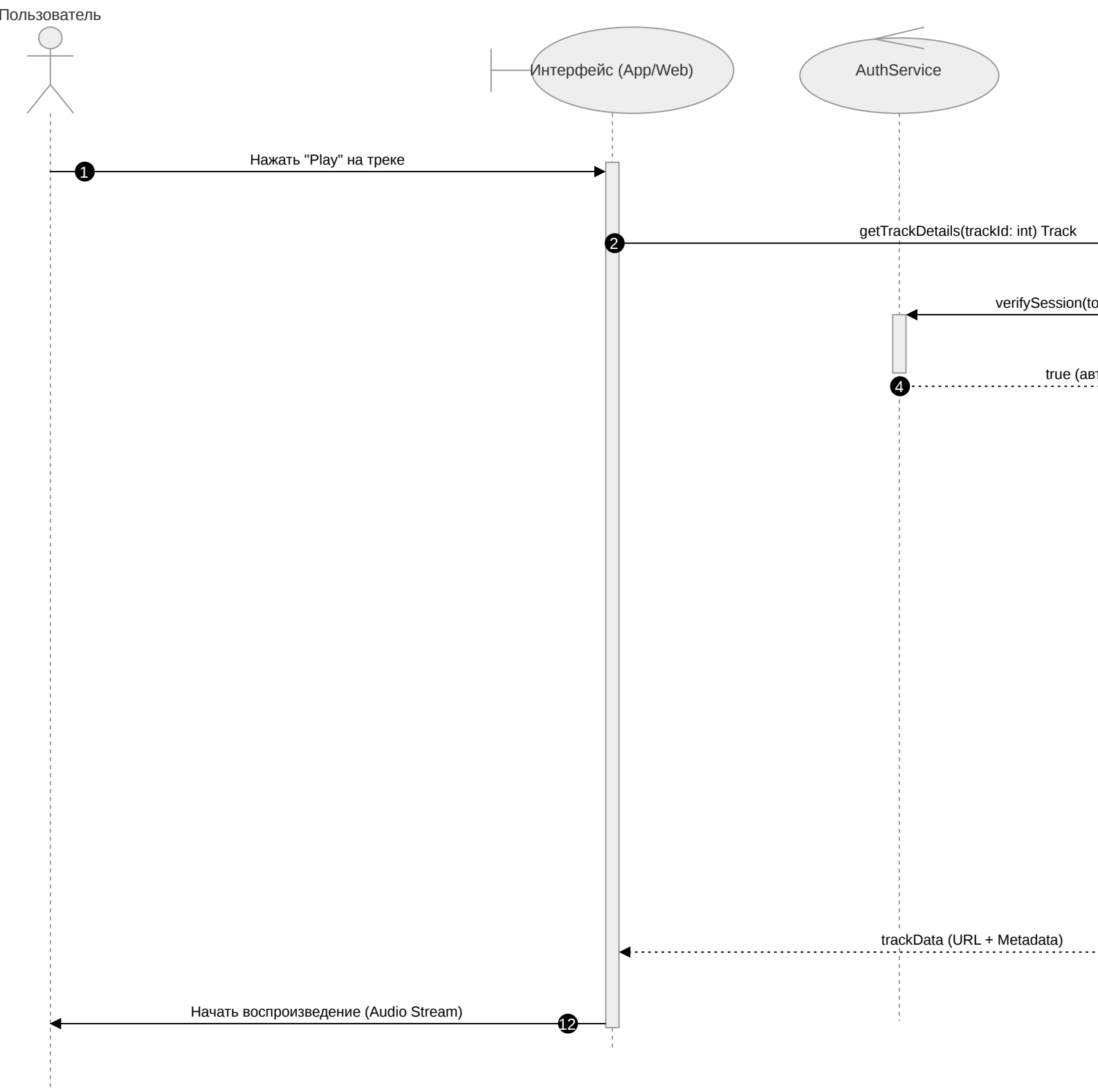

Провер
Проверка на права и кач

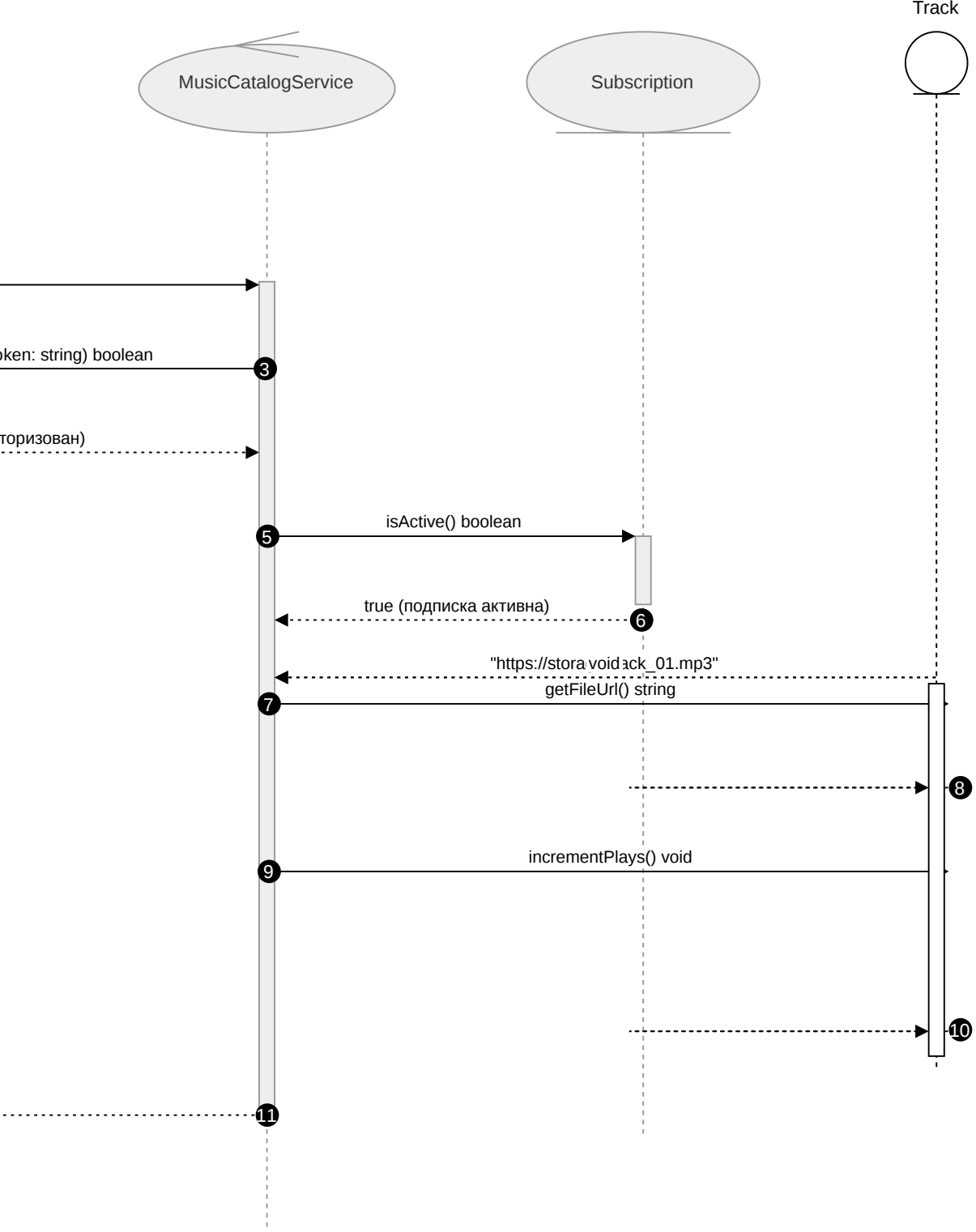

Загрузка файла
В подготовке (Draft)
Редактирование данных
Отправка на проверку
На модерации (Pending)
Отклонен (Rejected)
рка метаданных и прав
[Нарушение авторских прав или низкое качество]
Глобальное изменение метаданных
ество
[Удаление контент-менеджером]
[Проверка пройдена] / Присвоить статус Public
Опубликован (Published)
Нарушение правил (жалоба) / Удаление
Восстановление видимости
Скрытие трека пользователем
Удален (Deleted)
Скрыт (Hidden)


Запрос на прослушивание трека
Пользователь авторизован?
Да
Нет
Форма входа
Да
Нет
Установить стандартное качество 128kbps
Установить высокое качество 320kbps
Получить файл трека и метаданные
Подписка активна?
Начать стриминг аудиопотока
Записать лог прослушивания


Метод: incrementPlays
Обновить playCount в БД


Облачное хранилище / CDN
Аудио-контент (S3/MP3)
Audio Stream
Клиентское приложение (JS/Mobile)
S3 API / HTTPS
HTTPS / JSON
Music API Server (.NET/Java/Node)
Сервер приложений
Сервисы: Auth, Catalog, Payment
JDBC / SQL
БД (PostgreSQL/MySQL)
Платежный шлюз (API)


Пользователь
Интерфейс (App/Web)
AuthService
1
Нажать "Play" на треке
2
getTrackDetails(trackId: int) Track
verifySession(to
4
true (авт
trackData (URL + Metadata)
Начать воспроизведение (Audio Stream)
12


Track
MusicCatalogService
Subscription
oken: string) boolean
3
торизован)
isActive() boolean
5
true (подписка активна)
6
"https://stora void ack_01.mp3"
getFileUrl() string
7
8
incrementPlays() void
9
10
11